Valkyrie Package Documentation, version 1.0.0

James R. Swift, lordkekz, eltos, tinger <tinger@tinger.dev>, jneug, TimeTravelPenguin, NanamiNakano and RafDevX
Typst Community

September 05, 2026

Contents

1. Preface 4
    1.1. Use cases 4
2. Reading this manual 4
    2.1. Terminology 4
    2.2. Specific language 4
3. Examples 5
4. Data structures 5
    4.1. Schema 5
5. API Reference 5
    5.1. z.parse() 5
    5.2. z.base-type() 5
6. API Reference (Schema Definition) 5
    6.1. z.alignment() 5
    6.2. z.angle() 6
    6.3. z.any() 6
    6.4. z.array() 6
    6.5. z.boolean() 6
    6.6. z.bytes() 6
    6.7. z.choice() 6
    6.8. z.color() 7
    6.9. z.content() 7
    6.10. z.date() 7
    6.11. z.dictionary() 7
    6.12. z.direction() 7
    6.13. z.either() 8
    6.14. z.fraction() 8
    6.15. z.function() 8
    6.16. z.gradient() 8
    6.17. z.label() 8
    6.18. z.length() 8
    6.19. z.location() 9
    6.20. z.plugin() 9
    6.21. z.ratio() 9
    6.22. z.regex() 9
    6.23. z.relative() 9
    6.24. z.selector() 9
    6.25. z.string() 10
    6.26. z.stroke() 10
    6.27. z.symbol() 10
    6.28. z.tuple() 10
    6.29. z.version() 10
7. API Reference (Predefined schemas) 11
    7.1. z.schemes.papersize() 11
    7.2. author 11
8. API Reference (Assertions) 11

8.1. z.assert.contains() 11
8.2. z.assert.ends-with() 11
8.3. z.assert.eq() 11
8.4. z.assert.matches() 11
8.5. z.assert.max() 11
8.6. z.assert.min() 12
8.7. z.assert.neq() 12
8.8. z.assert.one-of() 12
8.9. z.assert.starts-with() 12
8.10. length 12
8.10.1. z.assert.length.equals() 12
8.10.2. z.assert.length.max() 12
8.10.3. z.assert.length.min() 13
8.10.4. z.assert.length.neq() 13
9. API Reference (Coercions) 13
9.1. z.coerce.array() 13
9.2. z.coerce.content() 13
9.3. z.coerce.date() 14
9.4. z.coerce.dictionary() 14
10. API Reference (Advanced) 15
10.1. z.z-ctx() 15
10.2. z.advanced.assert-base-type() 15

1. Preface

1.1. Use cases

The interface for a template that a user expects and that the developer has implemented are rearly one and the same. Instead, the user will apply common sense and the developer will put in somewhere between a token- and a whole-hearted- attempt at making their interface intuitive. Contrary to what one might expect, this makes it more difficult for the end user to correctly guess the interface as different developers will disagree on what is and isn't intuitive, and what edge cases the developer is willing to cover.

By first providing a low-level set of tools for validating primitives upon which more complicated schemas can be defined, `Valkyrie` handles both the micro and macro of input validation.

2. Reading this manual

2.1. Terminology

As this package introduces several type-like objects, this manual will denote these in a similar fashion as natively defined types. At present, these are:

schema to represent type-validation descriptions. Several are provided out-of-the-box both as individual units for discrete types which can be further composed, aswell as schemes defined within the library to serve a general community need.

z-ctx to represent the current state of the parsing heuristic. Modifying the values held in this type can significantly alter how certain aspects of the validation pipeline take place. There are very few cases where a template developer or consumer will need have any knowledge about it.

scope to represent an array of strings that keep track of the namespace within which a nested field is being validated. It's primary use is in error message generation.

Separately, some functions or arguments will be marked as internal, indicating that while these are available the end-user, they should be reserved for internal or advanced usage. The underlying implementation should not be relied upon and may be subject to change.

In general, users of this package will only need to be aware of the schema type (see Section 4.1)

2.2. Specific language

The key words **MUST**, **MUST NOT**, **REQUIRED**, **SHALL**, **SHALL-NOT**, **SHOULD**, **SHOULD NOT**, **RECOMMENDED**, **MAY**, and **OPTIONAL** in this document are to be interpreted as described in RFC 2119.

3. Examples

4. Data structures

4.1. Schema

5. API Reference

5.1. z.parse()

Interrogate an object with respect to a schema.

5.1.1. Parameters

```
parse(
  object: any,
  schemas: schema,
  ctx: z-ctx,
  scope: scope
)
```

5.1.1.1. object `any`

Object to interrogate.

5.2. z.base-type()

Schema generator. Provides default values for when defining custom types. name

5.2.1. Parameters

```
base-type(
  name: string,
  description: string,
  optional: bool,
  default: any,
  types: array,
  assertions: array,
  pre-transform: function,
  post-transform: function
) -> string: sdf
```

6. API Reference (Schema Definition)

6.1. z.alignment()

see #z.base-type()

6.1.1. Parameters

```
alignment() -> schema
```

6.2. z.angle()

see `#z.base-type()`

6.2.1. Parameters

```
angle() -> schema
```

6.3. z.any()

see `#z.base-type()`

6.3.1. Parameters

```
any() -> schema
```

6.4. z.array()

6.4.1. Parameters

```
array(
  name,
  assertions,
  min,
  max,
  ..args
)
```

6.5. z.boolean()

see `#z.base-type()`

6.5.1. Parameters

```
boolean() -> schema
```

6.6. z.bytes()

see `#z.base-type()`

6.6.1. Parameters

```
bytes() -> schema
```

6.7. z.choice()

see `#z.base-type()`

6.7.1. Parameters

```
choice(
  list,
  assertions,
  ..args
) -> schema
```

6.8. z.color()

see #z.base-type()

6.8.1. Parameters

```
color() -> schema
```

6.9. z.content()

see #z.base-type()

6.9.1. Parameters

```
content() -> schema
```

6.10. z.date()

see #z.base-type()

6.10.1. Parameters

```
date() -> schema
```

6.11. z.dictionary()

Valkyrie schema generator for dictionary types. Named arguments define validation schema for entries. Dictionaries can be nested.

6.11.1. Parameters

```
dictionary(
  dictionary-schema,
  name,
  default,
  pre-transform,
  aliases,
  ..args
) -> schema
```

6.12. z.direction()

see #z.base-type()

6.12.1. Parameters

```
direction() -> schema
```

6.13. z.either()

Valkyrie schema generator for objects that can be any of multiple types.

6.13.1. Parameters

```
either(
  strict,
  ..args
) -> schema
```

6.14. z.fraction()

see #z.base-type()

6.14.1. Parameters

```
fraction() -> schema
```

6.15. z.function()

see #z.base-type()

6.15.1. Parameters

```
function() -> schema
```

6.16. z.gradient()

see #z.base-type()

6.16.1. Parameters

```
gradient() -> schema
```

6.17. z.label()

see #z.base-type()

6.17.1. Parameters

```
label() -> schema
```

6.18. z.length()

see #z.base-type()

6.18.1. Parameters

```
length() -> schema
```

6.19. z.location()

see `#z.base-type()`

6.19.1. Parameters

`location() -> schema`

6.20. z.plugin()

see `#z.base-type()`

6.20.1. Parameters

`plugin() -> schema`

6.21. z.ratio()

see `#z.base-type()`

6.21.1. Parameters

`ratio() -> schema`

6.22. z.regex()

see `#z.base-type()`

6.22.1. Parameters

`regex() -> schema`

6.23. z.relative()

see `#z.base-type()`

6.23.1. Parameters

`relative() -> schema`

6.24. z.selector()

see `#z.base-type()`

6.24.1. Parameters

`selector() -> schema`

6.25. z.string()

Valkyrie schema generator for strings

6.25.1. Parameters

```
string(
  assertions,
  min,
  max,
  ..args
) -> schema
```

6.26. z.stroke()

see `#z.base-type()`

6.26.1. Parameters

```
stroke() -> schema
```

6.27. z.symbol()

see `#z.base-type()`

6.27.1. Parameters

```
symbol() -> schema
```

6.28. z.tuple()

Valkyrie schema generator for an array type with positional type reqruiements. If all entries have the same type, see `#z.array()` exact (bool): Requires a tuple to match in length

6.28.1. Parameters

```
tuple(
  exact,
  ..args
) -> schema
```

6.29. z.version()

see `#z.base-type()`

6.29.1. Parameters

```
version() -> schema
```

7. API Reference (Predefined schemas)

7.1. z.schemes.papersize()

7.1.1. Parameters

```
papersize() -> schema
```

7.2. author `schema`

8. API Reference (Assertions)

8.1. z.assert.contains()

Asserts that a tested value contains the argument (string)

8.1.1. Parameters

```
contains(value)
```

8.2. z.assert.ends-with()

Asserts that a tested value ends with the argument (string)

8.2.1. Parameters

```
ends-with(value)
```

8.3. z.assert.eq()

Asserts that tested value is exactly equal to argument

8.3.1. Parameters

```
eq(arg)
```

8.4. z.assert.matches()

Asserts that a tested value matches with the needle argument (string)

8.4.1. Parameters

```
matches(
  needle,
  message
)
```

8.5. z.assert.max()

Asserts that tested value is less than or equal to argument

8.5.1. Parameters

```
max(rhs)
```

8.6. z.assert.min()

Asserts that tested value is greater than or equal to argument

8.6.1. Parameters

```
min(rhs)
```

8.7. z.assert.neq()

Asserts that tested value is not exactly equal to argument

8.7.1. Parameters

```
neq(arg)
```

8.8. z.assert.one-of()

Asserts that the given value is contained within the provided list. Useful for complicated enumeration types.

- list (array): An array of inputs considered valid.

8.8.1. Parameters

```
one-of(list)
```

8.9. z.assert.starts-with()

Asserts that a tested value starts with the argument (string)

8.9.1. Parameters

```
starts-with(value)
```

8.10. length

8.10.1. z.assert.length.equals()

Asserts that tested value's length is exactly equal to argument

8.10.1.1. Parameters

```
equals(arg)
```

8.10.2. z.assert.length.max()

Asserts that tested value's length is less than or equal to argument

8.10.2.1. Parameters

```
max(rhs)
```

8.10.3. z.assert.length.min()

Asserts that tested value's length is greater than or equal to argument

8.10.3.1. Parameters

```
min(rhs)
```

8.10.4. z.assert.length.neq()

Asserts that tested value's length is not equal to argument

8.10.4.1. Parameters

```
neq(arg)
```

9. API Reference (Coercions)

9.1. z.coerce.array()

If the tested value is not already of array type, it is transformed into an array of size 1

```
#let schema = z.array(
  pre-transform: z.coerce.array,
  z.string()
)

#z.parse("Hello", schema) \
#z.parse(("Hello", "world"), schema)
```

```
("Hello",)
("Hello", "world")
```

9.1.1. Parameters

```
array(
  self,
  it
)
```

9.2. z.coerce.content()

Tested value is forcibly converted to content type

```
#let schema = z.content(
  pre-transform: z.coerce.content
)

#type(z.parse("Hello", schema)) \
#type(z.parse(123456, schema))
```

```
content
content
```

9.2.1. Parameters

```
content(
  self,
  it
)
```

9.3. z.coerce.date()

An attempt is made to convert string, numeric, or dictionary inputs into datetime objects

```
#let schema = z.date(
  pre-transform: z.coerce.date
)

#z.parse(2020, schema) \
#z.parse("2020-03-15", schema) \
#z.parse("2020/03/15", schema) \
#z.parse((year: 2020, month: 3, day: 15),
schema) \
```

```
datetime(year: 2020, month: 1, day: 1)
datetime(year: 2020, month: 3, day: 15)
datetime(year: 2020, month: 3, day: 15)
datetime(year: 2020, month: 3, day: 15)
```

9.3.1. Parameters

```
date(
  self,
  it
)
```

9.4. z.coerce.dictionary()

If the tested value is not already of dictionary type, the function provided as argument is expected to return a dictionary type with a shape that passes validation.

```
#let schema = z.dictionary(
  pre-transform:
z.coerce.dictionary((it)=>(name: it)),
  (name: z.string())
)

#z.parse("Hello", schema) \
#z.parse((name: "Hello"), schema)
```

```
(name: "Hello")
(name: "Hello")
```

- fn (function): Transformation function that the tested value and returns a dictionary that has a shape that passes validation.

9.4.1. Parameters

```
dictionary(fn)
```

10. API Reference (Advanced)

10.1. z.z-ctx()

This is a utility function for setting contextual flags that are used during validation of objects against schemas. Currently, the following flags are described within the API:

strict If set, this flag adds the requirement that there are no entries in dictionary types that are not described by the validation schema.

soft-error If set, this flag silences errors from failed validation parses. It is used internally for types that should not error on validation failures. See `#z.either()`

remove-optional-none This flag is used to indicate that, when parsing a dictionary, if a field is marked as optional in the schema, and its value is `none`, the key should be absent from the validated dictionary, otherwise, that it **SHOULD** be present in the validated dictionary with a value of `none`

10.1.1. Parameters

```
z-ctx(
  parent: dictionary default,
  ..flags: arguments
) -> z-ctx
```

10.1.1.1. parent dictionary or default

The parent context from which to derive. The parent context provides the default value of any missing flags

Default: `(:)`

10.1.1.2. ..flags arguments

Variadic contextual flags to set. Positional arguments are discarded.

10.2. z.advanced.assert-base-type()

10.2.1. Parameters

```
assert-base-type(
  arg,
  scope
) -> internal bool
```